

CHƯƠNG 4: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

Từ thiết kế triển khai, hệ thống được hiện thực thành một ứng dụng web chạy trên PHP và MySQL, kết hợp một service Python riêng cho chức năng đọc văn bản. Các trang PHP chịu trách nhiệm tạo giao diện, kiểm tra phiên đăng nhập và hiển thị dữ liệu; các endpoint trong api/ tiếp nhận request từ trình duyệt; nhóm hàm trong core/ dùng chung cho kết nối database, xác thực, hội thoại và ticket. Cách phân chia này giúp mỗi chức năng có một điểm xử lý rõ ràng nhưng vẫn giữ được sự gọn nhẹ của một ứng dụng PHP thuần.

Phần xây dựng tập trung vào ba nhóm việc: chuẩn bị môi trường chạy, hiện thực các thành phần frontend - backend - AI/NLP, và kết nối chúng thành các luồng sử dụng hoàn chỉnh. Người dùng có thể đăng nhập, xem dữ liệu cá nhân, đặt câu hỏi, nghe câu trả lời, đánh giá kết quả và gửi ticket. Ở phía quản trị, các chức năng kiểm duyệt tri thức, quản lý sinh viên, cập nhật lịch học và xử lý ticket được đặt trong cùng một dashboard.

4.1. Môi trường và công cụ phát triển

Dự án được triển khai theo mô hình web cục bộ, tương thích với các môi trường PHP như MAMP, WAMP hoặc XAMPP. Trong cấu hình mặc định của config.php, MySQL được trỏ tới 127.0.0.1 ở cổng 8889, cơ sở dữ liệu có tên do_an_udpm và tài khoản mặc định là root. Các giá trị này không bị cố định trong từng endpoint mà được đọc từ file .env, vì vậy có thể thay đổi theo máy chạy mà không phải sửa mã nguồn.

Thành phần	Công nghệ / công cụ	Vai trò trong triển khai
Web server và PHP	PHP thuần chạy trên Apache của MAMP/WAMP/XAMPP	Render các view PHP và xử lý request tại api/
Cơ sở dữ liệu	MySQL 8.0+, InnoDB, bộ ký tự utf8mb4	Lưu tài khoản, hồ sơ, học vụ, tri thức, chat và ticket
Frontend	HTML, CSS, JavaScript thuần	Tạo dashboard, chatbot, modal, lịch và các thao tác AJAX
AI/NLP	Google Gemini REST API và logic PHP	Phân loại câu hỏi, truy xuất context và sinh câu trả lời
TTS	Python, FastAPI và gTTS	Nhận văn bản tiếng Việt và trả file MP3 qua cổng nội bộ 8001
Quản trị dữ liệu	phpMyAdmin, script SQL và các script kiểm tra database	Import schema, dữ liệu mẫu và kiểm tra trạng thái bảng

Bảng 4.1. Môi trường và công cụ sử dụng trong quá trình xây dựng

4.1.1. Công nghệ Frontend (HTML, CSS, JavaScript)

Phần giao diện được viết trực tiếp trong các view PHP, sau đó kết hợp với HTML để tạo cấu trúc trang. views/login.php là điểm vào của quá trình xác thực; views/dashboard.php là trang làm việc của sinh viên; views/admin_dashboard.php là trang quản trị có nhiều tab. Cách render

phía máy chủ phù hợp với cách triển khai PHP hiện tại: dữ liệu hồ sơ, lịch học, điểm, học phí và thông báo được truy vấn trước khi phần HTML được gửi về trình duyệt.

CSS được tách theo phạm vi sử dụng. login.css định dạng khu vực tin tức và biểu mẫu đăng nhập; dashboard.css định dạng sidebar, thẻ thông tin, lịch, widget chatbot và các modal của sinh viên; admin.css định dạng sidebar quản trị, bảng dữ liệu, bộ lọc và cửa sổ phản hồi ticket. Việc tách file giúp thay đổi bố cục của một phân hệ mà không làm ảnh hưởng trực tiếp đến các phân hệ còn lại.

Tương tác phía trình duyệt chủ yếu nằm trong js/chatbot.js. File này quản lý lịch sử chat theo từng người dùng bằng localStorage, gửi câu hỏi qua fetch(), cập nhật câu trả lời vào khung hội thoại, hiển thị gợi ý, xử lý đánh giá và mở form ticket. Lịch sử ở trình duyệt chỉ giữ trạng thái hiển thị; phiên chat thực tế vẫn được backend ghi vào chat_sessions và chat_messages thông qua UUID do server tạo.

Khi trang dashboard được tạo, một đối tượng UTH_CONTEXT được đưa vào JavaScript để cung cấp các thông tin cần thiết cho giao diện như mã sinh viên, họ tên, ngày hiện tại, lịch hôm nay, lịch ngày mai, deadline và thông báo. Những dữ liệu này giúp chatbot hiển thị lời chào và gợi ý phù hợp, còn việc xác định dữ liệu cá nhân chính thức vẫn được thực hiện lại ở PHP theo session của người dùng.

```
const response = await fetch('../api/chatbot.php', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    message: text,
    sessionId: dbSessionUuid,
    mssv: ctx.mssv || ''
  })
});
```

Trích js/chatbot.js - gửi câu hỏi từ giao diện tới endpoint chatbot

Nội dung do người dùng nhập được xử lý trước khi đưa vào DOM. Ở phía PHP, các giá trị hiển thị trên dashboard dùng htmlspecialchars(); ở phía JavaScript, hàm escapeHTML() mã hóa các ký tự HTML cơ bản khi hiển thị câu hỏi của sinh viên. Câu trả lời của Bot có thể chứa một số thẻ định dạng có kiểm soát như strong hoặc br để giữ bố cục, vì vậy giao diện chỉ sử dụng cách render này ở khu vực đã được backend tạo ra.

[CHÈN HÌNH 4.1: Giao diện Dashboard Sinh viên với khu vực thông tin cá nhân, lịch và widget Chatbot]

4.1.2. Công nghệ Backend (PHP, MySQL)

Backend sử dụng PHP thuần, không phụ thuộc vào framework MVC. config.php đảm nhiệm việc đọc .env và cung cấp thông tin kết nối dùng chung. Các trang và endpoint đều require file cấu hình cùng core/faq_helpers.php, nhờ đó việc kết nối MySQL, chuẩn hóa chuỗi và truy vấn dữ liệu không bị lặp lại ở nhiều nơi.

```
loadEnvFile(__DIR__ . '/.env');
$apiKey = envValue('GEMINI_API_KEY', '');
$db_host = envValue('DB_HOST', '127.0.0.1');
$db_port = envValue('DB_PORT', '8889');
$db_name = envValue('DB_NAME', 'do_an_udpm');
```

Trích config.php - nạp cấu hình môi trường và thông tin kết nối

Nhóm hàm dbFetchAll(), dbFetchOne(), dbFetchValue() và dbExecute() trong core/faq_helpers.php đều chuẩn bị câu lệnh bằng PDO rồi mới truyền mảng tham số vào execute(). Cách làm này giữ câu SQL và dữ liệu nhập tách rời, phù hợp với các endpoint xử lý thông tin tài khoản, ticket và truy vấn cá nhân của sinh viên.

```
function dbFetchAll(PDO $pdo, string $sql, array $params = []): array {
    $stmt = $pdo->prepare($sql);
    $stmt->execute($params);
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}
```

Trích core/faq_helpers.php - hàm truy vấn dùng chung bằng PDO

Cơ chế xác thực được thực hiện bằng PHP Session. Sau khi appFindUserForLogin() tìm thấy tài khoản và password_verify() kiểm tra mật khẩu, login.php gọi session_regenerate_id(true), lưu user_id, username, role, họ tên và avatar vào \$_SESSION rồi điều hướng theo role. Các view tự kiểm tra role trước khi render: sinh viên vào dashboard.php, còn admin, staff và knowledge_reviewer vào admin_dashboard.php.

```
session_regenerate_id(true);
$_SESSION['user_id'] = (int)$user['id'];
$_SESSION['username'] = $user['username'];
$_SESSION['role'] = $user['role_code'];
$_SESSION['ho_ten'] = $user['full_name'];
```

Trích views/login.php - tạo phiên làm việc sau khi xác thực thành công

Các thao tác cập nhật nhiều bảng được xử lý trong transaction khi cần bảo đảm tính đồng bộ. Chẳng hạn, chức năng thêm lịch học của Admin lần lượt tạo hoặc tìm năm học, học kỳ, môn học, lớp học phần, buổi học và bản ghi enrollment; nếu có lỗi, transaction được rollback để tránh tình trạng dữ liệu chỉ được thêm một phần.

[CHÈN HÌNH 4.2: Sơ đồ các tầng xử lý từ views/ đến api/, core/ và MySQL]

4.1.3. Công nghệ AI/NLP (xử lý ngôn ngữ tự nhiên, TTS)

Xử lý câu hỏi được đặt trong api/chatbot.php. Trước khi truy vấn, appNormalizeText() chuyển chuỗi về dạng chữ thường, bỏ dấu tiếng Việt và gom khoảng trắng. classifyQuestion() dựa trên các nhóm từ khóa đã chuẩn hóa để nhận diện loại yêu cầu; detectEntities() tìm các thông tin như học kỳ, năm học, khóa sinh viên và mốc thời gian. Hai bước này giúp backend chọn đúng nhánh xử lý thay vì gửi mọi câu hỏi vào mô hình sinh văn bản.

Với câu hỏi về hồ sơ, điểm, lịch học, lịch thi hoặc học phí, hệ thống kiểm tra sinh viên trong session rồi truy vấn trực tiếp các bảng nghiệp vụ. Các hàm buildProfileReply(), buildGradesReply(), buildScheduleReply(), buildExamsReply() và buildTuitionReply() tạo câu trả lời từ dữ liệu database. Cách làm này giữ số liệu cá nhân ở phía máy chủ và không đưa điểm, lịch học hoặc công nợ riêng của sinh viên vào prompt Gemini.

Với câu hỏi về thông báo và deadline, buildAnnouncementReply() dùng Full-Text Search trên announcements và academic_deadlines, đồng thời lọc theo trạng thái công bố và thời gian hiệu lực. Với câu hỏi kiến thức chung, retrieveRagChunks() lấy các bài đã xác minh từ view v_active_verified_knowledge, nối với knowledge_chunks rồi dùng MATCH() của MySQL để

tìm ứng viên. Điểm truy xuất hiện tại kết hợp keyword score, metadata score, freshness score và priority boost; trường embedding_json được giữ trong schema nhưng chưa tham gia tính điểm vector.

Nếu có GEMINI_API_KEY, context đã chọn được gửi tới model gemini-2.5-flash-lite qua REST API generateContent. System prompt yêu cầu Bot chỉ trả lời dựa trên context được kiểm duyệt, không tự đoán dữ liệu học vụ và không lặp lại thông tin nhạy cảm. Khi không có khóa hoặc API gặp lỗi, hệ thống sử dụng nội dung các chunk đã được chọn để tạo câu trả lời dự phòng. Nhờ vậy, chatbot vẫn có thể phản hồi từ database ngay cả khi dịch vụ sinh văn bản bên ngoài không hoạt động.

```
$systemPrompt = "Bạn là ChatBot UTH. Chỉ trả lời dựa trên CONTEXT ".
                ."đã kiểm duyệt bên dưới. Nếu context không đủ, nói rằng chưa có ".
                ."thông tin đủ chính xác. Không tự đoán về dữ liệu học vụ.";
$answer = callGemini($apiKey, $userMessage, $systemPrompt);
```

Trích api/chatbot.php - ràng buộc nội dung trả lời bằng context đã kiểm duyệt

Chức năng đọc văn bản được tách thành service Python trong tts/server.py. FastAPI cung cấp GET /health để kiểm tra trạng thái và POST /api/tts để nhận văn bản. gTTS được gọi với lang='vi', tạo file MP3 tạm thời và trả về media type audio/mpeg. api/text_to_speech.php làm proxy cùng nguồn: kiểm tra JSON, chuyển text sang service ở 127.0.0.1:8001, sau đó chuyển nguyên audio về trình duyệt. js/chatbot.js nhận blob, tạo URL tạm để phát bằng Audio và dùng SpeechSynthesisUtterance làm phương án dự phòng.

```
@app.post('/api/tts')
async def text_to_speech(request: TextRequest) -> FileResponse:
    text = request.text.strip()[:4000]
    tts = gTTS(text=text, lang='vi', slow=False)
    tts.save(tmp_path)
    return FileResponse(tmp_path, media_type='audio/mpeg')
```

Trích tts/server.py - tạo file âm thanh tiếng Việt bằng gTTS

[CHÈN HÌNH 4.3: Sơ đồ luồng AI/NLP từ câu hỏi, phân loại, truy vấn dữ liệu, RAG đến Gemini và fallback]

4.2. Triển khai các tính năng chính

4.2.1. Tích hợp AI Bot vào hệ thống sinh viên

Chatbot được đặt trực tiếp trong dashboard.php thay vì tách thành một trang độc lập. Khi sinh viên mở cửa sổ Chat, giao diện hiển thị lời chào, một số câu hỏi gợi ý và ô nhập nội dung. Ba gợi ý ban đầu tập trung vào các nhu cầu có dữ liệu tương ứng trong hệ thống: xem kết quả học tập, xem lịch ngày mai và tra cứu học phí.

Dashboard đồng thời cung cấp các thẻ thao tác nhanh như đào tạo trực tuyến, hỗ trợ trực tuyến, cổng thanh toán, dịch vụ sinh viên, chương trình khung, đăng ký học phần, môn học điều kiện, tra cứu công nợ và kết quả học tập. Khi người dùng chọn một thẻ, JavaScript đưa câu hỏi mẫu vào khung chat và gọi cùng một luồng xử lý. Nhờ đó, giao diện giữ một điểm tương tác thống nhất thay vì phải xây dựng nhiều trang tra cứu riêng.

Khi sinh viên gửi câu hỏi, js/chatbot.js lưu câu hỏi vào lịch sử cục bộ theo một khóa owner riêng. owner được tạo từ userId hoặc MSSV nên lịch sử của người dùng này không bị trộn với

người dùng khác trên cùng trình duyệt. Backend trả sessionId và messageId trong JSON; sessionId được lưu lại để các câu hỏi tiếp theo nối vào cùng phiên chat, còn messageId được dùng khi sinh viên đánh giá hoặc báo lỗi câu trả lời.

Các thao tác sau câu trả lời được đặt ngay bên dưới tin nhắn Bot. Người dùng có thể nghe, sao chép, đánh dấu hữu ích, báo lỗi hoặc mở form gửi ticket. Khi câu hỏi không tìm thấy context đủ điểm, backend trả tín hiệu TICKET_OFFER; khi sinh viên chủ động yêu cầu hỗ trợ, backend trả TICKET_CONFIRM. JavaScript chuyển hai tín hiệu này thành nút xác nhận trong giao diện, tránh tạo ticket ngoài ý muốn.

```
if (botReply.includes('TICKET_OFFER')) {
  const cleanReply = botReply.replace(' TICKET_OFFER', '').trim();
  appendMessage(cleanReply, 'bot', null, true, botMeta);
  showTicketOffer(text);
}
```

Trích js/chatbot.js - chuyển tín hiệu fallback thành nút tạo ticket

Khi người dùng xác nhận, createRealTicket() gửi subject và description đến api/create_ticket.php. Endpoint kiểm tra role student, lấy student_id từ session, tạo mã theo dạng TK-YYYYMMDD-XXXX, ghi ticket chính và thêm tin nhắn đầu tiên vào ticket_messages. Nếu ticket được tạo từ một phiên chat hợp lệ, source_chat_session_id cũng được lưu để Admin có thể truy lại ngữ cảnh ban đầu.

[CHÈN HÌNH 4.4: Giao diện Chatbot trong Dashboard Sinh viên với câu hỏi, gợi ý, nút nghe, feedback và ticket]

4.2.2. Xây dựng logic xử lý Chat và trả lời tự động

Luồng xử lý bắt đầu bằng việc đọc JSON từ php://input và loại bỏ thông điệp rỗng. Sau khi kết nối database, chatbot xác định sinh viên từ user_id trong session; trường mssv trong request chỉ được dùng như một dữ liệu phụ trợ khi không có sinh viên trong session. Cách ưu tiên session giúp tránh việc người dùng tự thay đổi MSSV trong request để xem dữ liệu của tài khoản khác.

Tiếp theo, classifyQuestion() phân loại câu hỏi vào các nhóm tạo ticket, điều hướng portal, dữ liệu cá nhân, thông báo/deadline, trò chuyện thông thường hoặc kiến thức chung. detectEntities() bổ sung các điều kiện như học kỳ, năm học, khóa sinh viên và khoảng ngày. Kết quả phân loại cùng entities được lưu vào chat_messages để có thể theo dõi loại câu hỏi và nguyên nhân của một câu trả lời cụ thể.

Nếu câu hỏi thuộc nhóm dữ liệu cá nhân, backend yêu cầu sinh viên phải đăng nhập. Khi xác thực đủ điều kiện, match của PHP chọn hàm truy vấn phù hợp. Dữ liệu điểm lấy qua grades và enrollments; lịch học lấy qua class_schedule_sessions; lịch thi lấy qua exam_schedules; học phí lấy qua tuition_invoices; hồ sơ lấy qua student_profiles và programs. Nếu không có dòng dữ liệu phù hợp, hệ thống ghi nhận unanswered_questions và trả thông báo yêu cầu bổ sung thông tin thay vì tự tạo số liệu.

Nếu câu hỏi thuộc nhóm kiến thức chung, retrieveRagChunks() giới hạn số chunk theo system_settings, mặc định tối đa năm chunk và ngưỡng final score là 0,62. MySQL trước hết

chọn các ứng viên bằng MATCH() trên tiêu đề, từ khóa, nội dung bài viết và nội dung chunk. Sau đó PHP tính keyword overlap, điểm phù hợp metadata, freshness và priority. Mỗi ứng viên, kể cả ứng viên bị loại, được ghi vào rag_retrieval_logs cùng rank_position, final_score, selected_for_context và rejection_reason.

```
$finalScore = min(1.0, ($keywordScore * 0.85)
    + $metadataScore + $freshnessScore + $priorityBoost);
$isSelected = $finalScore >= $minFinalScore;

// selected_for_context và rejection_reason được ghi vào log
```

Trích api/chatbot.php - tính điểm và chọn context cho RAG

Các chunk đạt ngưỡng được ghép thành context gồm tên nguồn, độ tin cậy, điểm truy xuất, nội dung và đường dẫn. Khi có Gemini, context này được đưa vào system prompt trước khi gọi callGemini(). Câu trả lời nhận được được mã hóa các ký tự cần thiết rồi trả về cho trình duyệt. Khi không có khóa, Gemini trả lỗi hoặc trả về nội dung rỗng, backend chuyển sang database-rag và hiển thị trực tiếp các chunk đã kiểm duyệt.

Kết thúc mỗi nhánh, hàm finish() ghi tin nhắn assistant vào chat_messages và trả về JSON gồm reply, suggestions, sessionId, messageId, intent, intentClass và answerStatus. Nhờ có answerStatus, giao diện và Admin có thể phân biệt câu trả lời bình thường, thiếu context, bị chặn hoặc gặp lỗi. Khi không có context phù hợp, câu hỏi được đưa vào unanswered_questions và giao diện hiển thị lời mời tạo ticket để chuyển phần việc còn thiếu dữ liệu sang quy trình hỗ trợ.

[CHÈN HÌNH 4.5: Sơ đồ tuần tự xử lý một câu hỏi từ trình duyệt đến database, RAG/Gemini và phản hồi JSON]

4.2.3. Xây dựng trang Dashboard quản trị hệ thống (Admin)

admin_dashboard.php chỉ cho phép truy cập khi role trong session thuộc admin, staff hoặc knowledge_reviewer. Sau khi vượt qua bước kiểm tra quyền, trang kết nối database, bảo đảm view RAG có cấu trúc phù hợp rồi xác định tab hiện tại qua tham số GET tab. Các tab chính gồm Tổng quan, Hỗ trợ Tickets, Kiểm duyệt tri thức, Quản lý Sinh viên và Lịch sử Chat.

Tab Tổng quan lấy số lượng ticket, số ticket đang chờ xử lý, số ticket đã phản hồi, số bài tri thức verified, số bài pending, tổng phiên chat, tổng tin nhắn và câu hỏi chưa trả lời. Các số liệu được lấy trực tiếp bằng COUNT() từ tickets, knowledge_articles, chat_sessions, chat_messages và unanswered_questions. Bảng ticket gần đây dùng bộ lọc theo trạng thái để Admin ưu tiên những yêu cầu đang ở trạng thái open hoặc in_progress.

Quy trình cập nhật kho tri thức được chia thành hai bước. Admin hoặc người duyệt có thể thêm nguồn tại action add_source, sau đó thêm bài tri thức tại add_knowledge. Bài mới được tạo ở trạng thái pending và đồng thời có một dòng knowledge_chunks với chunk_index bằng 0. Khi nội dung được kiểm tra, action save_knowledge cập nhật bài và chunk; action verify_knowledge chuyển verification_status thành verified, ghi người duyệt, thời điểm duyệt và nâng confidence_level lên authoritative. View v_active_verified_knowledge chỉ lấy các bài đủ điều kiện, do đó dữ liệu nháp không đi thẳng vào câu trả lời của Bot.

```

if ($action === 'add_source') {
    // ghi nguồn ban hành vào knowledge_sources
} elseif ($action === 'save_knowledge'
    || $action === 'verify_knowledge') {
    // cập nhật knowledge_articles và knowledge_chunks
}

```

Trích views/admin_dashboard.php - rẽ nhánh các thao tác quản trị tri thức

Tab Quản lý Sinh viên cho phép thêm tài khoản và cập nhật hồ sơ. Khi thêm mới, hệ thống tìm hoặc tạo chương trình đào tạo, băm mật khẩu bằng password_hash(), ghi users rồi tạo student_profiles. Với lịch học, Admin nhập năm học, học kỳ, mã môn, tên môn, số tín chỉ, ngày trong tuần, giờ học, phòng và giảng viên. Backend kiểm tra khoảng thời gian, tạo các bản ghi cần thiết trong một transaction, thêm enrollment và cập nhật registered_count của lớp học phần.

```

$pdo->beginTransaction();
$academicYearId = adminFindOrCreateAcademicYear(...);
$semesterId = adminFindOrCreateSemester(...);
$subjectId = adminFindOrCreateSubject(...);
$sectionId = adminFindOrCreateCourseSection(...);
// thêm lịch học và enrollment
$pdo->commit();

```

Trích views/admin_dashboard.php - tạo dữ liệu lịch học theo transaction

Tab Hỗ trợ Tickets hiển thị mã ticket, sinh viên, tiêu đề, nội dung tóm tắt, trạng thái và thời gian tạo. Khi mở chi tiết, Admin xem các lượt trao đổi trong ticket_messages, gửi phản hồi qua action reply_ticket và có thể đóng ticket bằng close_ticket hoặc close_ticket_only. Phản hồi của Admin được ghi với sender_role là admin; trạng thái ticket chuyển sang waiting_student để sinh viên biết rằng yêu cầu đã được xử lý bước tiếp theo.

Tab Lịch sử Chat dùng dữ liệu chat_sessions, chat_messages, chat_feedback và rag_retrieval_logs để theo dõi chất lượng trả lời. Admin có thể xem câu hỏi, câu trả lời, trạng thái answer_status, feedback và ticket liên quan. Việc lưu cả kết quả được chọn lẫn kết quả bị loại trong log tạo cơ sở để kiểm tra vì sao Bot chọn một tri thức hoặc không tìm thấy context phù hợp.

Ở phía giao diện, trang Admin sử dụng sidebar cố định và cơ chế chuyển tab bằng JavaScript thay vì tạo nhiều trang quản trị riêng. Các form POST quay về admin_dashboard.php cùng tham số action; sau khi xử lý xong, PHP redirect về tab tương ứng. Cách làm này giữ mã nguồn tập trung, giảm số điểm điều hướng và giúp trạng thái thao tác của Admin dễ theo dõi trong một màn hình.

[CHÈN HÌNH 4.6: Admin Dashboard tại tab Tổng quan với các thẻ thống kê và danh sách ticket]

[CHÈN HÌNH 4.7: Màn hình Kiểm duyệt tri thức với trạng thái pending và verified]

[CHÈN HÌNH 4.8: Màn hình chi tiết ticket và luồng phản hồi giữa Sinh viên với Admin]

Sau khi triển khai, hệ thống hình thành một luồng hỗ trợ khép kín: sinh viên đăng nhập, tra cứu bằng chatbot, nghe TTS hoặc gửi ticket; Admin tiếp nhận yêu cầu, theo dõi hội thoại và cập nhật dữ liệu. Toàn bộ quá trình được lưu lại để thuận tiện kiểm soát và xử lý tiếp.